

## 2 Design of the line follower

In this section, we will build a controller to allow the robot to follow a reflective line taped on the floor using its infrared sensors. We will take the Pololu implementation as a basis and improve upon it.

The first step will be to implement a controller that can balance and go forward at a constant speed. Then, the infrared sensors will be used to compute the position of the reflective line relative to the center of the robot, and the controller will turn the robot accordingly. We will also build a model of the motors to gain more accurate control over the movement of the robot.

Figure 1 shows the overall structure that the software will follow. On every iteration, the sensors will be polled and their data will be used to compute an estimate of the state of the robot. This estimate will be compared to a set of reference signals to compute the error signals. Those reference signals represent the state that we wish the robot to be in. The controller will then take these errors and send appropriate commands to the motors to correct them. The controller will be a parallel PID controller.

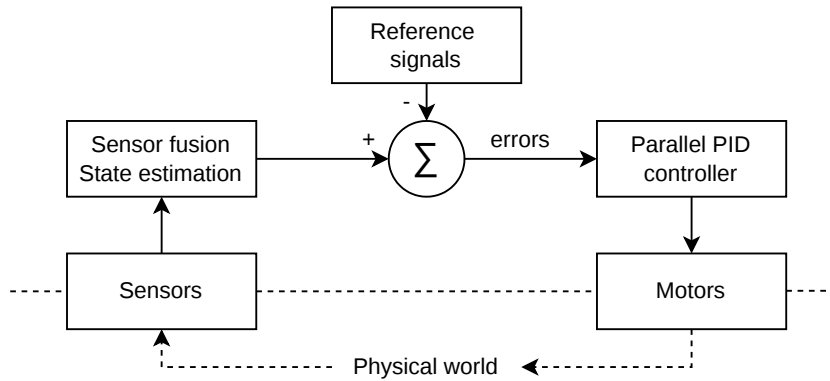


Figure 1: Update loop of the robot.

### 2.1 Controller design

Two-wheeled self-balancing robots can be modeled after an inverted pendulum. If we wish to balance it, we can use a controller to correct the errors on the balancing angle and the position as well as their derivatives only by actuating the motors.

This section presents the three errors that the controller should take of and a scheme of the controller is shown in figure 2. Section 2.2 goes into detail on how to compute the state of the robot as well as the reference signals.

- **Balancing angle:** For the robot to be at equilibrium, its center of mass should be above the contact points of its wheels with the floor. We will first predict the balancing angle using its current derivative, and then, compare it to the reference angle of the robot (its equilibrium angle when it is not accelerating) to get the error signal. Multiplying the error signal by a constant will give us a command to accelerate the robot in the direction it is leaning toward in order to counter that error.

- **Distance traveled:** For the robot to travel at a set speed or to stand still, we need to compute how far it has traveled and compare that to the desired distance traveled. The reference value can be obtained using a speed profile. A PD controller will handle the error on the distance traveled using its proportional term, and the error on the speed using its derivative term.
- **Line position:** To follow the line, the robot will be going forward, and keep itself centered on the line. If its sensors see the line on its left side, then the robot should turn left before continuing forward. A proportional controller is enough for this purpose. If we knew the curvature of the line to be smooth, we could add a derivative term to predict where the line will be relative to the robot on the next iteration.

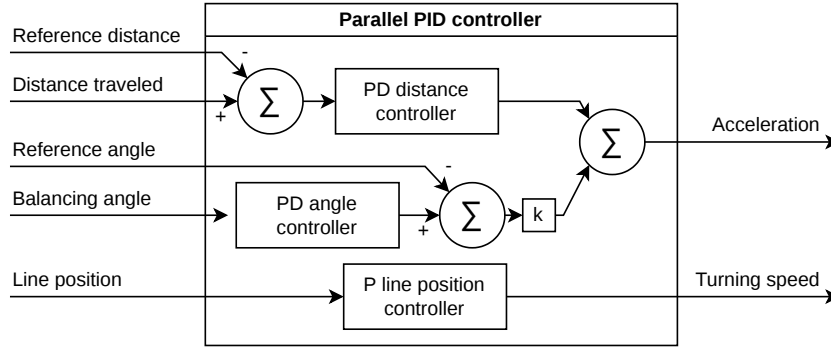


Figure 2: Parallel PID controller of the line follower.

## 2.2 State estimation

### 2.2.1 Balancing angle

The balancing angle can be estimated from the accelerometer and the gyroscope. The gyroscope gives us the angular speed and the accelerometer gives us the direction and amplitude of the combined forces applied on the robot, including gravity.

The accelerometer is prone to high frequency noise. On top of that, it will not only give the direction of gravity, which would allow us to compute the balancing angle of the robot, but the direction of the sum of all the forces on the robot. If the robot accelerates or rotates, the estimate will not be as accurate.

We can reduce the effect of those perturbations by passing the output of the accelerometer through a low-pass filter. The filter will average out those values over a window of time, and that average value will be more representative of the direction of gravity. These measurements will give us a good estimate of the balancing angle over a long enough period of time.

$$low\_pass\_filter(Acc, k) = \frac{1}{window\_size} \sum_{i=0}^{window\_size-1} Acc_{k-i}$$

The gyroscope is much more accurate, and gives us the angular speed of the robot. Integrating the values of the gyroscope will give us the current angle of the robot. The

integrating process, however, will add small measurement errors over time which will give us a more and more erroneous estimate of the balancing angle even just after a few seconds.

Passing the accelerometer and gyroscope estimates through a complementary filter that mostly trusts the gyroscope will solve our problem. It will have the accuracy of the gyroscope over short periods of time while maintaining the estimate accurate over longer periods of time by skewing it toward the average value of the accelerometer.

$$angle_k = (1 - \epsilon) \cdot (angle_{k-1} + Gyro_k) + \epsilon \cdot low\_pass\_filter(Acc, k)$$

Once we have computed the estimate of the balancing angle, we can compute the error and send a corresponding acceleration command to the motors to fix it.

$$command_k = k_p \cdot (angle_k + \Delta t \cdot angle'_k - reference\_angle_k)$$

### 2.2.2 Distance traveled

The motors are equipped with encoders that measure the amount the wheels have rotated since startup. By averaging out the counts on both encoders, we have an estimate of the distance traveled by the robot. We can now keep the robot balancing in place or move it with a given speed using a reference signal.

### 2.2.3 Line sensing

Using Pololu's reflectance sensors, we can estimate the position of the reflective line below the robot. Five sensors are aligned in an array. Each sensor is composed of a pair of emitter (an infrared LED) and receptor (a phototransistor). Upon being polled, the emitters will send out infrared light and the receptors will respond with a value representing the amount of time it took them to fully charge. The value returned ranges from 0 to 2500µs, at which point the function will timeout. This can happen when the sensors are above a non-reflective surface.

Although Pololu uses calibration in their example code, the sensors seem to be pretty much identical to each other in terms of the values they measure. The main difference comes from the arrangement of the sensors in the array and their distance to each other. The middle sensor and the two sensors to its side are close to each other (9.5mm apart each) while the two sensors on the extremities lie 28.5mm away from the center. This results in the middle sensor giving off a much smaller value than the others as it receives not only the infrared emitted by its own emitter but also those of the sensors next to it.

For our application, that of following a line by keeping the robot centered on top of it, this can be ignored as we only wish to know whether the line is to the left or to the right of the robot, and have an indication of how far from the center it is.

To compute the position of the line, we use the following formula in which we apply a min-max scaling to the values of the line sensors before computing a weighted average. The weighted average will pair the distances of the sensors from the center with the

values they read. This will give us the position of the line relative to the center which we can use as our error signal.

$$line\_position = \sum_{sensor \in Sensors} position_{sensor} * \frac{value - min\_value}{max\_value - min\_value}$$

## 2.3 Motors

Two different actions can be taken using the motors. We can accelerate both wheels by the same amount to accelerate the robot forward or backward. Or we can add a certain speed to the wheels in different directions to make the robot turn. Which action should be taken depending on the measured errors is described above in section 2.1.

In our design of the controller, we assumed the commands given by the controllers to be proportional to the speed of the motors. This is not really the case, even though the robot can still balance and move with this linearity assumption. In section TODO, we develop a model of the motors and use it to make the robot move more smoothly. The model used has different impacts on the behavior of the robot.

## 2.4 Initialisation phase

So far, we've only talked about how the robot should behave to follow the line. This assumes the robot is already following the line.

When we raise the robot, it will not have an error that is representative of the errors it gets when normally operating. Even though it is resilient to external perturbations (up to a certain point, of course), it would be convenient if the robot could stay in place for a second and not rotate around itself before it actually starts following the line. We have thus added an initialisation phase in which the robot corrects its orientation based on the difference between the encoder values instead of the line position error.

$$orientation\_error = k * (L - R)k = circle\_angle = \frac{encoderssize}{radiusrobot}$$

This is like the P-controller for the path tracker which we will explain in the next section.

### 3 Path-tracking

A path tracker is an algorithm that will follow a reference point that is generated using code. At any point in time, the generated trajectory will give us the reference state of the robot.

#### 3.1 Design of the PID controller

Unlike with the line follower, the robot isn't simply moving along a physical line. We thus need to consider its position on the xy-plane. We split the position into two different errors: the forward distance error and the lateral distance error. That also means, as the robot can only go in the direction that its wheels are facing, that its orientation matters.

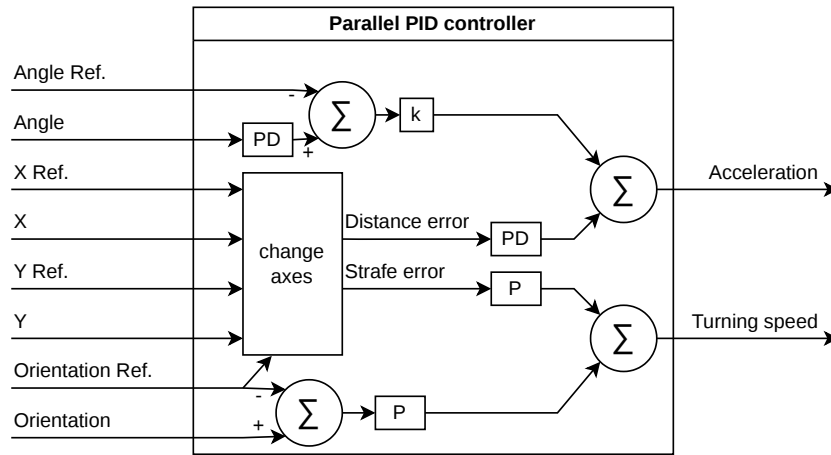


Figure 3: Scheme of the reference point tracking controller.

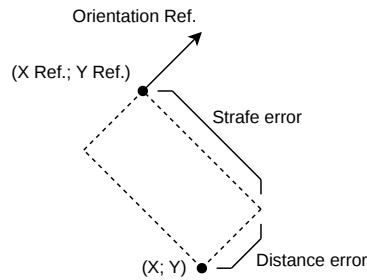


Figure 4: Computation of the forward and lateral distance errors.

The different errors that the controller needs to address are as follows:

- **Balancing angle:** This is the same as for the line follower. We want to keep the robot mostly upright, but its angle will also affect its acceleration. A PD controller will be used to accelerate the robot.

- **Orientation error:** The robot will only ever go forward. When the robot has to move, it should be headed in the direction where the reference position will be on the next iteration. This error takes care of rotating the robot in the right direction. The robot is able to turn very quickly and without overshooting so a proportional controller is enough.
- **Forward distance error:** This error represents how far forward or backward the robot is compared to the reference point. The forward/backward distance computation is based on the reference orientation. A PD controller is used for the same reasons described in the line follower. We wish to control how far and how fast the robot is going.
- **Lateral distance error:** To fix a lateral distance error while moving forward, the robot can get back on track by turning toward the reference path. Lateral distance errors occur for different reasons like an error in orientation on the previous iterations or a difference in the speed of the motors. We use a proportional controller to turn the robot while still maintaining its forward movement along the reference path. Lateral distance errors can also occur when the robot is balancing while stationary, we further discuss lateral distance errors in section TODO.

## 3.2 State estimation

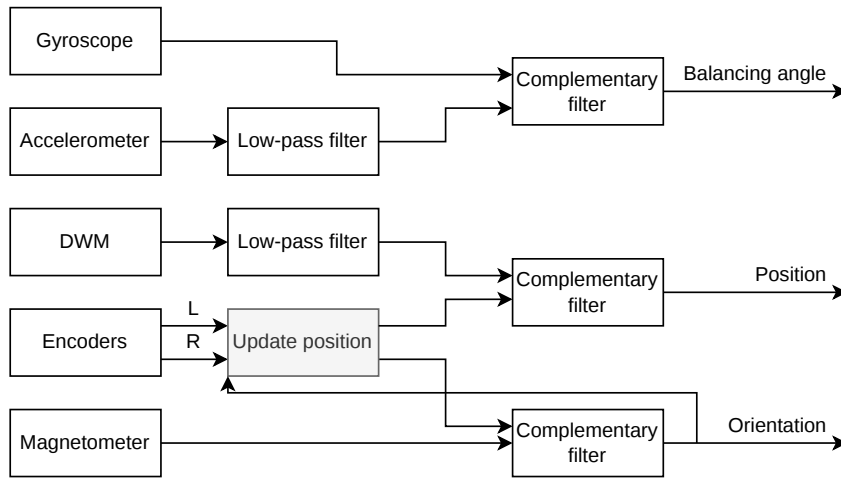


Figure 5: Sensor fusion and state estimation procedure.

The estimate for the balancing angle will be the same as for the line follower.

The estimate for the position and orientation of the robot can be done using the encoders only. This will be explained in the next section.

In the same way that small errors add up when only using the gyroscope, a drift will appear if we only use the encoders for estimating the position of the robot. We will need to use the magnetometer and the DWM to fix the orientation and position errors respectively.

### 3.2.1 Position and orientation using the encoders

At every iteration, the wheels will have rotated a certain amount. We can update the position ( $x, y$ ) and orientation ( $\theta$ ) of the robot based on the values read from the encoders.

If the change in value of the left encoder ( $L$ ) is equal to the change of value of the right encoder ( $R$ ), we assume that the robot has moved in a straight line. If  $L$  and  $R$  are different, then we assume that the robot has moved in a circular arc.

---

**Algorithm 1** Update the position of the robot using only the encoders

---

**Require:** the current state of the robot  $(x, y, \theta)$  and the new encoder readings  $(L, R)$ , robot\_width is the distance between the two wheels of the robot

**Ensure:**  $x, y$  and  $\theta$  are updated using  $L$  and  $R$

```
if  $L = R$  then                                     ▷ Moving in a straight line
     $\Delta x \leftarrow L * \cos(\theta)$ 
     $\Delta y \leftarrow L * \sin(\theta)$ 
     $\Delta \theta \leftarrow 0$ 
else                                                  ▷ Moving and turning in the arc of a circle
     $\Delta \theta \leftarrow (R - L) / \text{robot\_width}$           ▷ Convert the distance into radians
     $\text{circle\_radius} \leftarrow L / \Delta \theta + \text{robot\_width} / 2$ 
                                                    ▷ Radius of the circle traced by the center of the robot
     $\Delta x \leftarrow \text{circle\_radius} * (-\sin(\theta) + \sin(\theta + \Delta \theta))$ 
     $\Delta y \leftarrow \text{circle\_radius} * (\cos(\theta) - \cos(\theta + \Delta \theta))$ 
end if
 $x \leftarrow x + \Delta x$ 
 $y \leftarrow y + \Delta y$ 
 $\theta \leftarrow \theta + \Delta \theta$ 
```

---

### 3.2.2 Removing the drift

Although the encoders alone give good results over short periods of time, they are subject to drift. The state of the robot is updated using complementary filters that mostly trust the encoders. The  $x$  and  $y$  coordinates are updated using the DWM readings and the orientation angle is updated using the magnetometer. For an explanation on how to get the heading of the robot from the magnetometer, see annex TODO.

### 3.2.3 Initialisation of the state

Since the magnetometer and DWM deal in absolute values, we need to initialise the state to a correct value. We've added an initialisation phase to get the absolute heading and position of the robot once it is stable and upright.

Here are the steps of the initialisation in order:

- On startup of the program, the robot is laying down and the gyroscope is calibrated.
- It is now ready to be raised up.
- Once raised, the robot balances in place (by setting all its reference signals to zero) and waits for a set amount of time to pass, then considers itself stable.

- It then polls the DWM and magnetometer repeatedly and averages the values.
- The robot now knows its absolute position and heading, it can start following its predefined reference signals.

If the sensors are not well calibrated, this method of initialising the state can create an offset between the state estimated and the actual position of the robot. This problem will be showcased in section TODO.

### 3.3 Trajectory generation

A trajectory is a pair of a path and a speed profile.

A path is a continuous two-dimensional curve. A parametric curve can be used as a path. By reparameterizing the curve, we can get a mapping from the distance traveled along the curve to a position in the plane.

A speed profile is a mapping from time or distance that gives out a speed at that time or distance along the path. A speed profile can also give the current acceleration or even jerk at a set time.

To generate a trajectory, we will first need to generate a path, and then generate a speed profile that is linked to that path. We want to move the robot along a set of checkpoints, so we will start by moving the robot in straight lines, and then explore the use of splines and parametric curves in general.

#### 3.3.1 Straight lines

The path for a straight line segment is defined by the start and end points. To get a point at a certain distance along the path, we can linearly interpolate between the two points based on the distance traveled.

To travel along several straight line segments, we repeat two steps for however many line segments are in the path. First, the robot should rotate so that its wheels are aimed in the direction of the end point. And then, the robot simply needs to go forward.

As for the speed profile, we have several options. A constant speed along the path would be the simplest one. Given  $MS$ , the maximum speed at which the robot can go, we can set the speed of the robot to  $MS$  for an amount of time equal to the length of the segment divided by  $MS$ . The problem with this constant speed profile is that the acceleration and deceleration are very sudden and make the robot much less stable.

A trapezoidal speed profile uses a maximum acceleration  $MA$  and can be divided in three parts for a straight line segment.

- First phase: the robot accelerates until it reaches its maximum speed or until it has traveled half of the total distance of the segment.
- Second phase: the robot cruises at a constant speed of  $MS$ . This phase might not exist in the case where the distance to travel is too short, and so the robot reaches the halfway mark before reaching its maximum speed.
- Third phase: the robot decelerates in order to reach its final destination with a speed of zero.



The acceleration is now bounded and the robot can behave more smoothly. This trapezoidal speed profile is what is used in a lot of two-wheeled self-balancing robots. The resulting acceleration profile can be used as a reference signal to modify the balancing angle of the robot.

$$reference\_balancing\_angle(t) = equilibrium\_balancing\_angle + k * acceleration\_profile(t)$$

We can go one step further. The balancing angle of the robot changes continuously, so sending a signal that changes abruptly does not take into account this physical limit of the system. Using an S-curve, a speed profile generated with a bounded jerk, we can get better results. The formulas for generating such a profile, even just for a straight line, become unwieldy. They are described in full in [TODO]. Thanks to S-curves, the reference signal for the balancing angle is now continuous.

### 3.3.2 Parametric curve trajectories

**Choice of splines** As our main objective is to make the robot move through checkpoints, we have decided to use cubic splines to create a path for the robot. They give us enough freedom to define the start and end position and heading of the robot.

Hermite splines are defined using a start and end point as well as the derivative of the path at the start and end point. For our purposes, we only really care about the heading of the robot at the endpoints, but we still have to define a magnitude. For a single segment from a point  $A$  to a point  $B$ , we will define the final heading as the orientation describe by the  $\vec{AB}$  vector. We will set the magnitude of the end derivatives to half the length between the two points as it is stated in [TODO] that it gives reasonable results.

Centripetal Catmull-Rom splines allow us to define a spline without using derivatives. They are defined using four points and create a path linking the two middle points. Their downside is that, in a more complex path with several checkpoints, they can only create a path between the second and the next to last point. We will be using Hermite splines on those two path segments to fill in the gaps. We use the derivatives defined by the Catmull-Rom splines on the second and next to last points, and define the derivatives as described in the previous paragraph for the first and last points.

If we wish to define checkpoints along with desired orientations, we can fall back to using Hermite splines on the whole path.

**TODO Speed profile generation** (not coded yet)

## 4 Results of the line-follower

### 4.1 Calibrating the PID values for balancing

The PID controller needs to be given some constants to work properly. In this section, we tune the PID parameters and explore what happens when the values are not properly tuned. We will do the same in the section 5.1 for the path-tracker.

We will first tune the parameters so that the robot can balance upright. As explained in the code written by Pololu, the first step TODO read pololu documentation (alpha relative D for prediction) (alpha P for strength of acceleration) (distance D for not oscillating wildly out of control) (distance P for to stay in place) Figure 6 shows the robot balancing in place after this calibration.

Figure 6: Line follower balancing in place.

After adding a speed profile, we can now take care of the line following P-controller. At every iteration of the control loop, the robot will check whether the line is to its left or to its right. Then, it will rotate toward the line. And finally, thanks to the speed profile, it will go forward to get closer to the line. Figure 7 shows the calibrated P-controller in action.

Figure 7: Line follower following a line. The red line is drawn using the data gathered by the robot, it is representative of where the line is relative to the robot, but might not be up to scale.

From this graph, we can see two interesting behaviors that stem from the same cause. The first one is that when the line is constantly curving in the same direction (to the right between  $x=250$  and  $x=300$ ), the robot tends to stick on the outside of the curve. The second is that when the curvature of the line changes (between  $x=0$  and  $x=100$ ), the robot crosses the line and goes beyond it by quite a bit before being able to reorient itself properly. This is because the position of the line has an effect on the orientation of the robot. The orientation doesn't cause a punctual change like lateral movement would, it affects the future of the robot as well. This behavior is reminiscent of an integral controller in the sense that it creates an overshoot in the error. Adding a derivative term to this controller might be a good idea.

It is also important to note that this P-controller was calibrated using a specific constant speed to follow the line. If the proportional constant is too big, the robot will turn so strongly that its orientation will not be aligned with the line anymore and it will not be able to turn in time to correct itself. If the constant is too small, the robot will wander off the line before being able to make a turn. The same logic applies if the speed of the robot is too small (resp. big) because it will make the proportional constant relatively too big (resp. small).

### 4.2 Faster initialisation with equilibrium angle

In the previous section, we showed that the robot was able to follow a line. We will now focus on the initialisation phase of the robot.

On startup and when not using the DWM, the robot considers its initial position to be  $(0; 0)$ . In figure 7, we can see that the robot actually goes backward by about 20cm before following the line forward. This is because the reference angle and the equilibrium angle are not matching.

The controller will notice an error in the balancing angle of the robot. That error will cause the controller to accelerate the robot backward. A new error in terms of the distance will occur and accelerate the robot forward. At equilibrium, these two errors cancel each other out thanks to our parallel PID controller. The robot is then stable, balancing around its equilibrium angle, and a bit backward compared to where it is supposed to be.

After setting the initial reference angle value to this newly found balancing angle at equilibrium, the robot becomes stable much faster after being risen. Also, it doesn't move far away from its initial position anymore.

This equilibrium angle can change if the floor is tilted, or not perfectly flat, if the wheels aren't perfectly round or if weight is added to the robot. The parallel PID controller can adapt to these changes at the cost of a distance error. But, given that the equilibrium angle is quite far from  $0^\circ$  usually, setting the reference angle to this new value has helped in making the robot more stable on initialisation.

### **4.3 Model of the motors**

maybe doing pt-balancer and then model of the motors is better to show lateral distance errors being handled

## 5 Results of the path-tracker

### 5.1 Calibrating the PID values for balancing

The balancing angle controller and distance controller can be taken from the line follower. The only difference with the distance controller is that the state estimation signal is computed differently. This does not have an impact on the results as the robot remains aligned with its path, thus both computations give similar distance signals.

The lateral distance controller and the orientation controller both affect the turning speed of the robot. As both of them are P-controllers, this means that we can describe one in terms of the other. It is more intuitive to transform the lateral distance error into an orientation error. Arbitrarily, we chose that when the lateral distance error was equal to the width of the robot (the distance between both its wheels), it would turn half a radian toward the path. So, for a lateral distance error of about 10cm (depending on which specific robot we use), the robot will turn about  $28.4^\circ$ . If the controller were weaker, it would take too long to correct the error, and if it were too strong, it might jitter wildly when small lateral disturbances are met.

We are now only left with the calibration of the orientation controller. According to the physical model of the robot, turning it is less complex than balancing or accelerating it. The only constraints we have to take care of when it comes to rotating the robot is the speed at which we do so. We use speed profiles to make sure these constraints are respected. Thus, we can increase the proportional constant of the orientation controller as much as possible to make it more responsive. That being said, if we make this constant too big, precision errors from the motors, encoders and magnetometer could will make the robot strongly jitter. Figure 8 shows these behaviors. We use  $k_p = 10$  in our controller.

### 5.2 Straight lines

Now that our controller is calibrated, we can test it on straight line segments. Figure 9 shows the robot reaching some predefined checkpoints. At every checkpoint, it will rotate toward the next and then go forward until it reaches its destination. A delay has been added between every action the robot takes to give it time to stabilize.

When the robot reaches a checkpoint, it then overshoots a bit before coming back toward the checkpoint. To decelerate properly, the robot needs to tilt backward, meaning that it needs to accelerate forward. In doing so, it will go beyond its reference distance signal, explaining the overshoot.

When fully stopped, not far enough when turning leads to lateral distance error when rotating

lateral errors are fixed as expected, the robot doesn't have many hiccups when on straight segments (curves will probably cause more issues...)

### 5.3 One segment exp.

let's look at the behavior in the time domain, instead of space domain of pt\_square

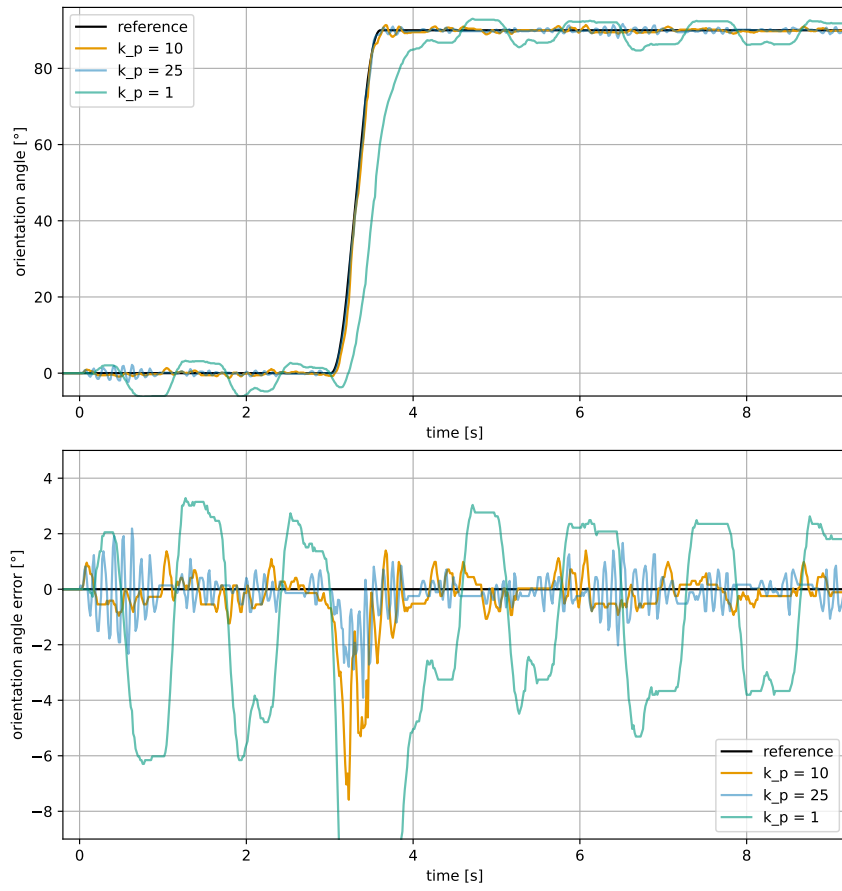


Figure 8: Path tracker rotating in place using different proportional constants for the orientation angle controller.

## 5.4 Speed profile comparisons

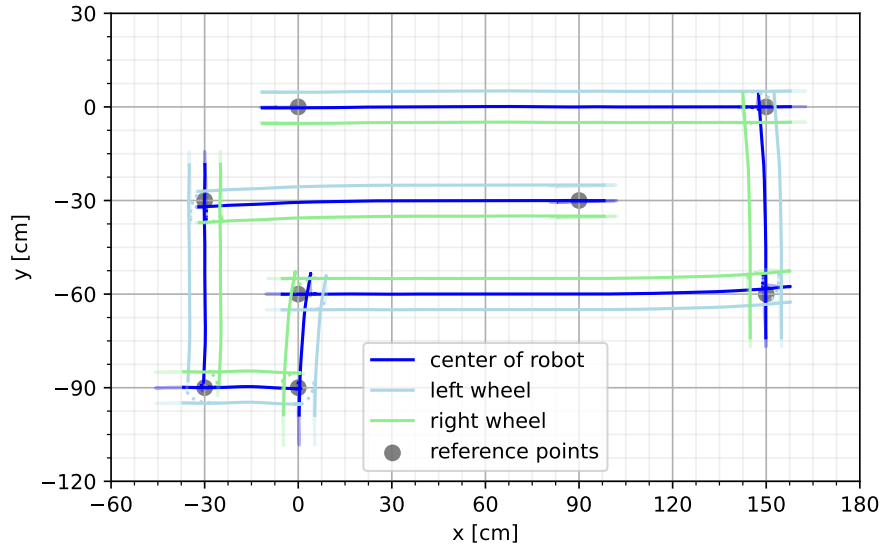


Figure 9: Path tracker following checkpoints using straight lines. The robot starts at (0, 0).

## 6 nothing

### 6.1 nothing

#### 6.1.1 Balancing: Finding alpha zero through experiment

The following figure 10 shows two things: - sse on distance on a slightly slanted floor (relative strength of pid constants on alpha error and distance error)

Figure 10: x, y, alpha of robot when balancing

#### 6.1.2 One Turn: Finding max rotational speed through experiment

TODO: one-turn experiment instead of one-segment experiment to find max rot. speed + speed profile comparisons

#### 6.1.3 One Segment: Finding max acc, speed, max ref alpha through experiment

Using S-curves (everyone says they're better but don't implement them, they're tedious to implement) TODO: compare with rectangular acceleration profile (and rectangular speed profile?)

Figure 11: state and errors of robot when moving forward by 2 meters

#### 6.1.4 Effect of the DWM and Magnetometer on the behavior of the robot

**DWM and Magnetometer active** guess: no drift, some more noise, but drift slow enough that slow fusion works, so small noise

Figure 12: state and errors of robot when moving forward by 60 centimeters

Figure 13: Perceived position of the robot while using or not the magnetometer and Decawave modules

**DWM only** guess: slow drift in theta will make the robot think that moving toward x will be fine, but it will move on a different axis, causing huge trouble, especially as the fusion with dwm is slow (maybe not fall off table)

**Magnetometer only** doesn't drift in theta anymore when going in a square or a circle, it drift in position much more than it does when magnetometer is off guess: magnet has errors that differ from (also, noisy from accelerometer as well as magnetometer)

## **7 Additional notes and improvements on the robot**

### **7.1 Adding integral PID constants to remove SSE on distance, ...**

### **7.2 Better motors**

### **7.3 Effect of the battery charge**

- The magnetometer (hard/soft-iron distortions?) calibration changes bias by a lot (the sphere is offcentered by a different amount at different battery charge levels)
- The wheels are more responsive/stronger -i better motors didn't work as well after fully charging the batteries